

category: dotnet-general

tags: [json-snapshot, cache-staleness, dto-sync, device-controller, register-flow]

severity: medium

created: 2026-08-05

updated: 2026-08-05

project-origin: App-Access-V2 (iAccessDesktopv2.Avalonia)

Cache JSON-snapshot trong controller không đồng bộ với AppState → UI hiện GUID/rỗng thay vì dữ liệu thật

Tình huống gặp phải

Đang làm gì? Tính năng gì? Môi trường nào?

Dialog "Danh sách đăng ký" (`RegisteredUsersWindow`, chuột phải node thiết bị) trong `iAccessDesktopv2.Avalonia` —

hiển thị danh sách khách hàng + định danh (thẻ/vân tay) đã đăng ký trên thiết bị, đọc từ `MemoryCollection` app-side join với `CustomerDto.AccessKeys` để lấy mã/tên thẻ thật.

Triệu chứng / Lỗi

Sau khi user "Hủy" rồi "Đăng ký lại" một thẻ trên thiết bị, mở lại dialog "Danh sách đăng ký": tên/mã khách hàng vẫn đúng, nhưng dòng định danh của thẻ vừa đăng ký lại hiện GUID kỹ thuật (`be5e9325-991f-4a8b-93a2-fea6a6...`) thay vì mã thẻ thật, và loại định danh hiện "Không xác định" thay vì "Thẻ". Các định danh cũ (không bị hủy/đăng ký lại) vẫn hiển thị đúng.

Nguyên nhân gốc rễ (Root Cause)

Hai lỗi cộng hưởng:

- Thiếu field khi cập nhật cache in-memory:** sau khi đăng ký thành công, `MainViewModel` thêm `AccessKey` mới vào `customer.AccessKeys` (cache `AppState.Customers`) nhưng chỉ gán `Id/CustomerId/Type` — BỎ SÓT `Code/Name` dù dữ liệu này đã có sẵn trong request (`RegisterUserControllerRequest.AccessKeyCode/AccessKeyName`). Khi UI join theo `AccessKey.Id`, `Code` rỗng → fallback hiển thị GUID.
- Cache JSON-snapshot bị đóng băng:** `KztekControllerBase.Customers` (field cache riêng của controller thiết bị) chỉ được gán lại (`Customers = JsonConvert.DeserializeObject<...>(customersJson)`) tại lần gọi `RegisterUserAsync(requestJson, customersJson)` gần nhất — đây là một BẢN SAO deserialize từ JSON, KHÔNG phải tham chiếu sống tới `AppState.Customers`. Nếu không có lệnh

đăng ký/xóa nào khác chạy sau đó trên CHÍNH thiết bị này, field này giữ nguyên snapshot cũ mãi mãi (kể cả sau khi `AppState.Customers` đã được cập nhật ở nơi khác) — vì vậy method đọc (`GetRegisteredUsers()`, không tham số) không bao giờ thấy được thay đổi mới nếu chỉ sửa lỗi #1.

Bài học tổng quát: khi 1 lớp giữ cache bằng cách **deserialize JSON tại thời điểm gọi API** (parity pattern IPC — request/response qua JSON để giữ layer thuần), cache đó là **snapshot đông cứng**, không tự đồng bộ với state gốc. Bất kỳ method READ nào dùng lại cache đó về sau đều có nguy cơ đọc dữ liệu cũ nếu không có cơ chế refresh chủ động.

Giải pháp

1. Bổ sung `Code = data.AccessKeyCode, Name = data.AccessKeyName` khi tạo `newAccessKey` VÀ khi cập nhật `existingAccessKey` trong success-handler của `MainViewModel` (không chỉ gán `Type/CustomerId`).

2. Đổi method READ (`GetRegisteredUsers()`) từ "không tham số, dựa cache nội bộ" sang "nhận tham số

JSON tươi tại thời điểm gọi" — cùng pattern JSON contract đã dùng cho `RegisterUserAsync`:

```
```csharp
```

```
// Interface (Abstractions — thuần net8.0, không kéo domain type CustomerDto):
```

```
IEnumerable<RegisteredUserInfo> GetRegisteredUsers(string customersJson);
```

```
// Implementation: deserialize tại chỗ, fallback về cache cũ nếu JSON rỗng/lỗi
```

```
var customersSource = TryDeserialize(customersJson) ?? Customers;
```

```
`
```

3. Call site (nơi mở dialog) truyền `JsonConvert.SerializeObject(AppState.Customers)` — luôn là

dữ liệu MỚI NHẤT tại đúng thời điểm user bấm mở dialog, không phụ thuộc lần đăng ký gần nhất.

## Áp dụng lại (How to reuse)

- Khi thấy 1 class giữ field cache được gán lại bằng `JsonConvert.DeserializeObject<T>(paramJson)` bên trong 1 method ghi (write/command) → PHẢI hỏi: "field cache này có bị method READ khác dùng lại không, và method đó có được gọi ĐỘC LẬP với method ghi không?" Nếu có → cache đó CÓ THỂ stale.
- Khi thêm 1 object mới vào list cache dùng cho join/display (VD: `customer.AccessKeys.Add(...)`), liệt kê ĐẦY ĐỦ property object đó cần cho MỌI nơi join tới nó (không chỉ property đang cần ngay lúc viết code) — kiểm tra chéo với property mà UI/display code thực sự đọc (`Code, Name` ở đây).
- Ưu tiên: method READ nhận tham số dữ liệu mới tại thời điểm gọi (giống contract JSON đã có sẵn trong codebase) thay vì để READ tự ý dùng field cache nội bộ — đặc biệt trong kiến trúc có ranh giới JSON giữa các layer (Abstractions thuần net8.0 không được reference domain type).

## Chú ý / Cạm bẫy (Gotchas)

- ⚠ Đừng tưởng sửa lỗi #1 (thêm `Code/Name`) là đủ — nếu không sửa luôn lỗi #2 (cache đóng băng),

bug vẫn tái hiện trong trường hợp KHÔNG có lệnh đăng ký nào khác chạy sau đó trên cùng thiết bị.

- ⚠️ `Customers = JsonConvert.DeserializeObject<List<CustomerDto>>(customersJson)` tạo ra object graph HOÀN TOÀN MỚI — không phải reference tới list gốc. Mutate list gốc sau đó (`AppState.Customers`) KHÔNG bao giờ phản ánh vào field đã gán trước đó.
- ⚠️ Khi đổi signature 1 method của interface dùng chung (`IMemoryCollectionProvider`), phải grep TOÀN BỘ call site (kể cả những chỗ chỉ dùng `is IMemoryCollectionProvider` để enable/disable nút, không gọi method) để không bỏ sót nơi cần cập nhật.

## Tham chiếu

- Project liên quan: `iAccessDesktopV2.Avalonia` (App-Access-V2)
- Files: `ViewModels/Main/MainViewModel.cs`,  
`iAccess.Devices.Kztek/KztekControllerBase.cs`,  
`iAccess.Devices.Abstractions/IMemoryCollectionProvider.cs`,  
`Views/Main/DeviceTreePanelView.axaml.cs`